```
$i = 0;
$size = count($myArr);

for ( ; $i < $size; ) {
    print "{$myArr[$i]} is here.\n";
    $i++;
}
```

Empty expressions might not appear to be useful with the trivial examples here, but there are use-cases for them. Imagine a scenario where complex logic might be required to determine if a loop should end; trying to fit complex tests into a `for` expression could well make for unreadable code.

Skipping Iterations

Sometimes it is useful to exclude certain elements from the loop operation. Perhaps the data type is incompatible with a certain function, or the value is of no interest. To force a loop to jump to the next iteration before the end of the block, you can use the construct `continue`:

```
$albums = [
    ['title' => 'Number of the Beast', 'artist' => 'Iron Maiden'],
    ['title' => 'Whatever', 'artist' => 'Some other band'],
    ['title' => 'Wasted Years', 'artist' => 'Iron Maiden'],
];

foreach ($albums as $album)  {
    if ($album['artist'] !== 'Iron Maiden') {
        continue;
    }

    print "{$album['title']} is an Iron Maiden album.\n";
}

Output:
Number of the Beast is an Iron Maiden album.
Wasted Years is an Iron Maiden album.
```